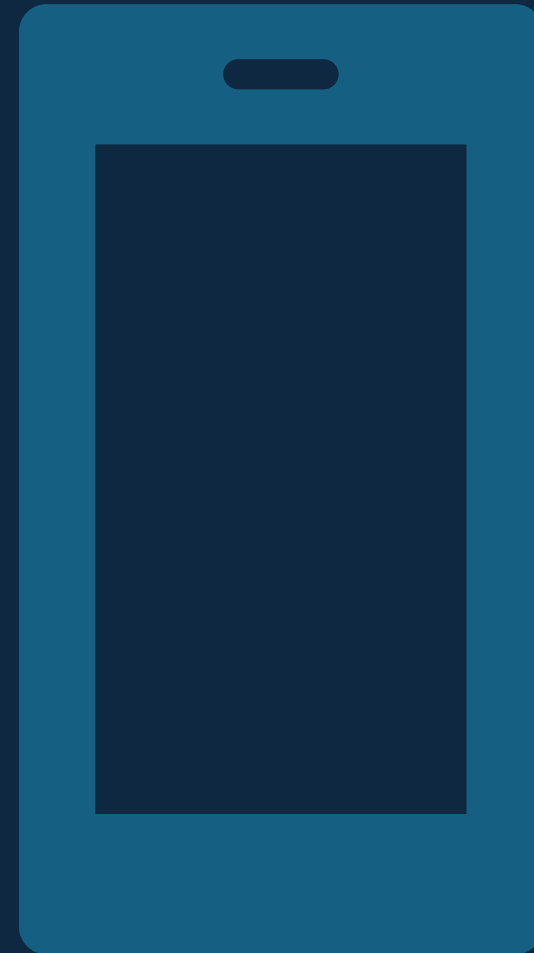


Navegação, Integração e Persistência

Material

MOBILE_007





O Cenário (Case Study)

- Desenvolveremos o "**CineApp**", um aplicativo simples composta por 3 telas:
 - **Home:** Exibe uma lista de filmes vindos de uma "API" (simulada).
 - **Detalhes:** Ao clicar em um filme, abre uma nova tela com a sinopse e um botão para "Salvar nos Favoritos".
 - **Favoritos:** Uma tela que lista os filmes salvos na memória do celular, persistindo mesmo se fechar o app.

Catálogo de Filmes (CineApp)

- O case escolhido é um "**Catálogo de Filmes**". É um cenário clássico que permite demonstrar perfeitamente:
 - **Listagem (Home)**: Consumo de dados (API).
 - **Detalhes (Navegação + Parâmetros)**: Passar o ID do filme de uma tela para outra.
 - **Favoritos (Persistência)**: Salvar o filme localmente no celular (AsyncStorage).

CINEAPP: Estrutura da Aplicação Mobile

(Case Study: Unidade 3 - Navegação, Integração e Persistência)



Conceitos-Chave: API Integration → Stack Navigation & Params → Local Storage (AsyncStorage)

Preparação do Ambiente

- No powershell ou cmd digite:
`npx create-expo-app cineapp -t blank`
- Entre na pasta cineapp, e instale as dependências.
Navegação
`npm install @react-navigation/native @react-navigation/native-stack`
- # Dependências do Expo para navegação
`npx expo install react-native-screens react-native-safe-area-context`
- # Persistência de Dados (Storage)
`npx expo install @react-native-async-storage/async-storage`



Bibliotecas:

@react-navigation/native

@react-navigation/native-stack

- ➡ **@react-navigation/native:** É o "cérebro" da navegação. Ele gerencia o histórico de telas (quem veio antes, para onde vai).
- ➡ **@react-navigation/native-stack:** É o tipo de navegação em Pilha. É o padrão do Android/iOS, onde uma tela entra por cima da outra com uma animação de deslizar.

Dependências Nativas

- **npx expo install** (IMPORTANTE): Diferente do npm install puro, esse comando verifica qual versão do Expo você está usando e baixa versões dessas bibliotecas que são 100% compatíveis. Isso evita erros de versão que travam o app.
- **react-native-screens**: Faz o React usar as telas nativas do sistema operacional para renderizar, economizando muita memória e bateria. Sem isso, a navegação fica lenta.
- **react-native-safe-area-context**: Sabe aquele "recorte" (notch) da câmera no iPhone ou nos Androids novos? Essa biblioteca garante que o título do seu app não fique escondido atrás da câmera ou da barra de bateria.

Banco de Dados Local

- O **@react-native-async-storage/async-storage** é um sistema de armazenamento de dados **persistente**, **simples**, **descriptografado** e **assíncrono**, global para todo o aplicativo.
- Uma analogia é:
 - "O AsyncStorage é o equivalente ao localStorage do navegador, mas rodando nativamente no celular."
- Ele armazena dados no formato **Chave-Valor** (Key-Value).

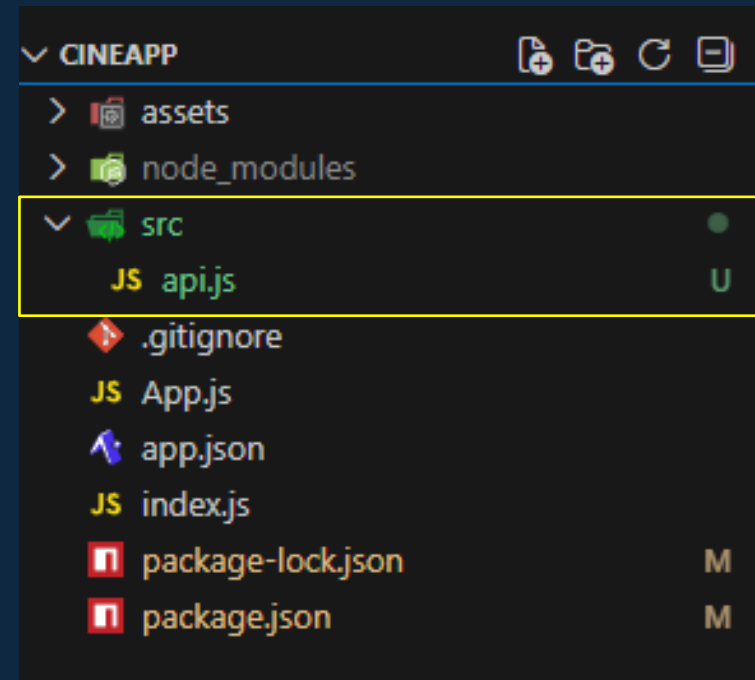
Implementação

- Vamos dividir o código em **Serviços** (lógica de dados) e **Telas** (Interface).



A. Camada de Dados (Simulando API e Storage)

- Crie um arquivo chamado **api.js** na raiz (ou pasta **src**).
- Aqui simulamos o Fetch API ou Axios. Em um app real, substituiríamos o `setTimeout` por uma chamada HTTP real.

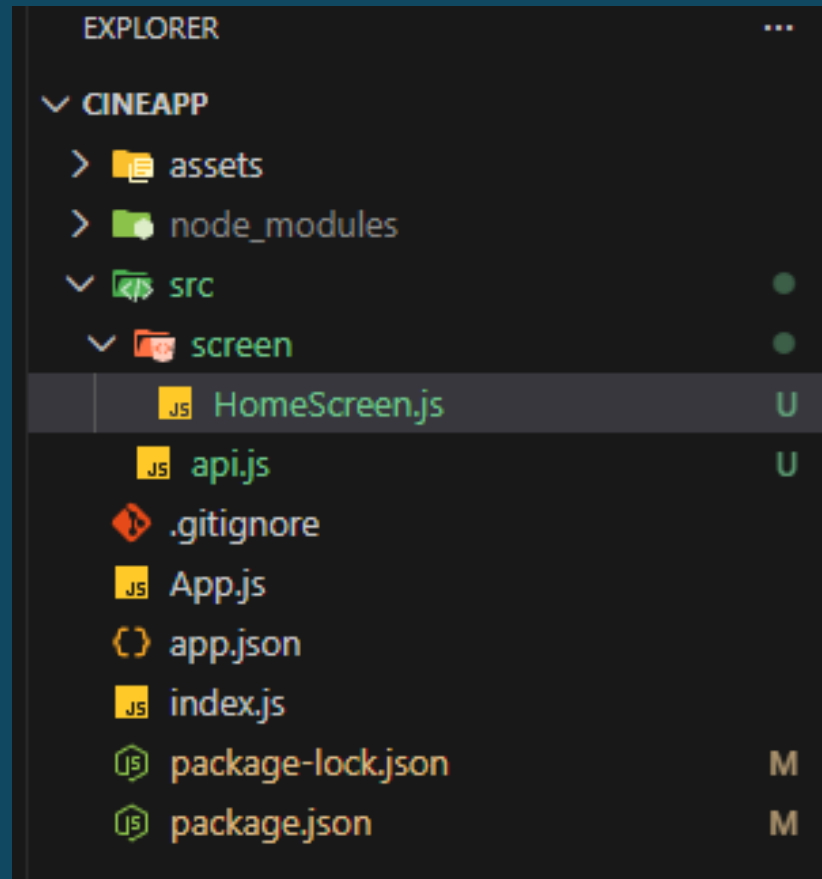


```
// src/api.js
const FILMES MOCK = [
  { id: '1', titulo: 'Matrix', ano: 1999, sinopse: 'Um programador descobre a verdade sobre a realidade.' },
  { id: '2', titulo: 'Interestelar', ano: 2014, sinopse: 'Exploradores viajam por um buraco de minhoca no espaço.' },
  { id: '3', titulo: 'O Poderoso Chefão', ano: 1972, sinopse: 'O patriarca de uma dinastia do crime organizado.' },
];

// Simula uma chamada assíncrona (como se fosse buscar na internet)
export const buscarFilmes = () => {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(FILMES MOCK);
    }, 1000); // Demora 1 segundo para "carregar"
  });
};
```

B. Tela Principal: Lista de Filmes

➔ Arquivo: src/screens/HomeScreen.js





```
1 // src/screen/HomeScreen.js
2 import React, { useEffect, useState } from 'react';
3 import { View, Text, FlatList, TouchableOpacity, StyleSheet, ActivityIndicator, Button } from 'react-native';
4 import { buscarFilmes } from '../api'; // Importando nossa API simulada
5
6 export default function HomeScreen({ navigation }) {
7   const [filmes, setFilmes] = useState([]);
8   const [loading, setLoading] = useState(true);
9
10  // useEffect substitui o "OnCreate" para carregar dados iniciais
11  useEffect(() => {
12    async function carregarDados() {
13      const dados = await buscarFilmes(); // Consumo da API (Assíncrono)
14      setFilmes(dados);
15      setLoading(false);
16    }
17    carregarDados();
18  }, []);
19
20  if (loading) {
21    return <ActivityIndicator size="large" color="#0000ff" style={{flex: 1}} />;
22  }
```



```
1  return (  
2    <View style={styles.container}>  
3      <Button title="Ver Meus Favoritos" onPress={() => navigation.navigate('Favorites')} />  
4  
5      <FlatList  
6        data={filmes}  
7        keyExtractor={(item) => item.id}  
8        renderItem={({ item }) => (  
9          <TouchableOpacity  
10             style={styles.card}  
11             // AQUI OCORRE A PASSAGEM DE PARÂMETROS  
12             // Passamos o objeto inteiro 'item' para a próxima tela  
13             onPress={() => navigation.navigate('Details', { filme: item })}  
14           >  
15             <Text style={styles.titulo}>{item.titulo}</Text>  
16             <Text style={styles.ano}>{item.ano}</Text>  
17           </TouchableOpacity>  
18         )}  
19       />  
20     </View>  
21   );  
22 }
```



```
1  const styles = StyleSheet.create({
2    container: { flex: 1, padding: 20, backgroundColor: '#f0f0f0' },
3    card: { backgroundColor: '#fff', padding: 15, marginBottom: 10, borderRadius: 8, elevation: 2 },
4    titulo: { fontSize: 18, fontWeight: 'bold' },
5    ano: { color: '#666' }
6  });
```

Link para o código:

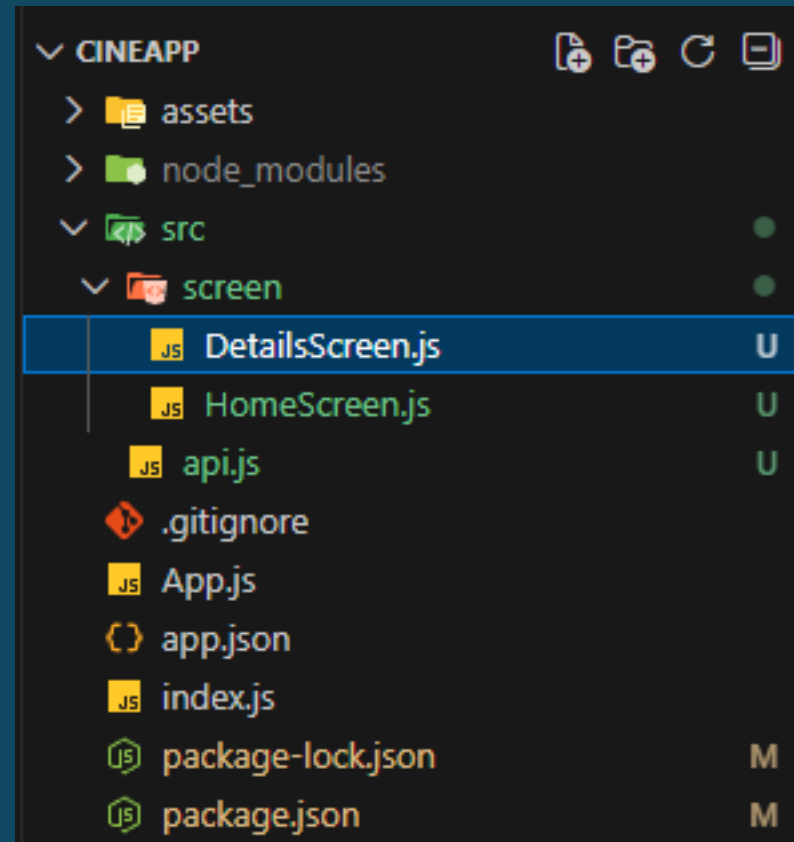
<https://drive.google.com/file/d/1A8YdcxbA2btxJtEhNkLryYGIB4uKFCSx/view?usp=sharing>

Componentes Utilizados

Componente	Analogia Web (HTML)	Função no CineApp
View	div	Segurar o layout
Text	p, span, h1	Exibir Título e Ano
FlatList	ul + li (turbinado)	Listar os Filmes com rolagem
TouchableOpacity	a (link) ou button	Detectar o toque para ir aos Detalhes
ActivityIndicator	Gif de Loading	Avisar que está carregando

C. Tela de Detalhes: Recebendo Parâmetros e Salvando

- Arquivo: `src/screens/DetailsScreen.js`



```
1 // src/screen/DetailsScreen.js
2 import React from 'react';
3 import { View, Text, Button, Alert, StyleSheet } from 'react-native';
4 import AsyncStorage from '@react-native-async-storage/async-storage';
5
6 // Recebemos "route" para acessar os parâmetros enviados pela Home
7 export default function DetailsScreen({ route, navigation }) {
8   // Desestruturação para pegar o parâmetro "filme"
9   const { filme } = route.params;
10
11   // Função para Persistir Dados (AsyncStorage)
12   const salvarFavorito = async () => {
13     try {
14       // 1. Ler o que já existe salvo
15       const favoritosAtuais = await AsyncStorage.getItem('@cinefatec_favoritos');
16       let listaSalva = favoritosAtuais ? JSON.parse(favoritosAtuais) : [];
17
18       // 2. Verificar se já não está salvo
19       const jaExiste = listaSalva.find(f => f.id === filme.id);
20       if (jaExiste) {
21         Alert.alert("Aviso", "Este filme já está nos favoritos!");
22         return;
23       }

```

```
1 // 3. Adicionar o novo e salvar
2 listaSalva.push(filme);
3 await AsyncStorage.setItem('@cinefatec_favoritos', JSON.stringify(listaSalva));
4
5 Alert.alert("Sucesso", "Filme salvo nos favoritos!");
6 navigation.goBack(); // Volta para a tela anterior
7
8 } catch (e) {
9   Alert.alert("Erro", "Não foi possível salvar.");
10 }
11 };
12
13 return (
14   <View style={styles.container}>
15     <Text style={styles.tituloGrande}>{filme.titulo}</Text>
16     <Text style={styles.detalhe}>Ano: {filme.ano}</Text>
17     <Text style={styles.sinopse}>{filme.sinopse}</Text>
18
19     <View style={{ marginTop: 20 }}>
20       <Button title="Salvar nos Favoritos" onPress={salvarFavorito} />
21     </View>
22   </View>
23 );
24 }
```



```
1  const styles = StyleSheet.create({  
2    container: { flex: 1, padding: 20 },  
3    tituloGrande: { fontSize: 28, fontWeight: 'bold', marginBottom: 10 },  
4    detalhe: { fontSize: 16, color: 'blue', marginBottom: 20 },  
5    sinopse: { fontSize: 16, lineHeight: 24, fontStyle: 'italic' }  
6  });
```

Código disponível em:

<https://drive.google.com/file/d/1cTH9rXX1TsTjr5iivzSWBBzpxzsojEvH/view?usp=sharing>

Componentes no DetailsScreen.js

➤ 1. Componentes Visuais (UI)

➤ Alert (O Pop-up Nativo)

- **Função:** Exibe uma caixa de diálogo nativa do sistema (Android ou iOS) que se sobrepõe a tudo.
- **No código:** Usamos `Alert.alert("Título", "Mensagem")`.

➤ Button (O Botão Padrão)

- Na Home, usamos `TouchableOpacity` (personalizado). Aqui usamos o componente `<Button />` cru.
- **Característica:** Ele renderiza o botão com a aparência **padrão do sistema operacional**.
- **Limitação:** Ele é difícil de estilizar (não aceita `style` livremente, apenas a prop `color`). Usamos aqui pela simplicidade didática de criar uma ação de "Salvar".

Componentes no DetailsScreen.js

➤ 2. Objetos de Navegação e Dados

- Embora não sejam "componentes visuais" (tags), estes são os elementos lógicos novos que fazem a mágica acontecer nesta tela.

➤ `route.params` (O "Correio")

- **O Conceito:** A tela Home "enviou" um pacote de dados (o objeto filme). A tela Details precisa "abrir" esse pacote.
- **No código:** `const { filme } = route.params;`
- **Explicação:** O objeto `route` contém informações sobre a rota atual. A propriedade `params` guarda tudo que foi passado via `navigation.navigate()`. É assim que a tela sabe *qual* filme exibir (Matrix ou Interestelar) sem precisar fazer uma nova consulta à API.

➤ `navigation.goBack()` (O Retorno Programático)

- **O Conceito:** O React Navigation já cria uma seta "Voltar" no topo da tela automaticamente. Porém, às vezes queremos forçar o retorno via código.
- **No código:** Logo após salvar o filme e dar o Alert, chamamos `navigation.goBack()`.

Componentes no DetailsScreen.js

➡ 3. A Lógica de Persistência (Revisão Prática)

➡ JSON.stringify vs JSON.parse

➡ No DetailsScreen, aplicamos a regra de ouro do AsyncStorage:

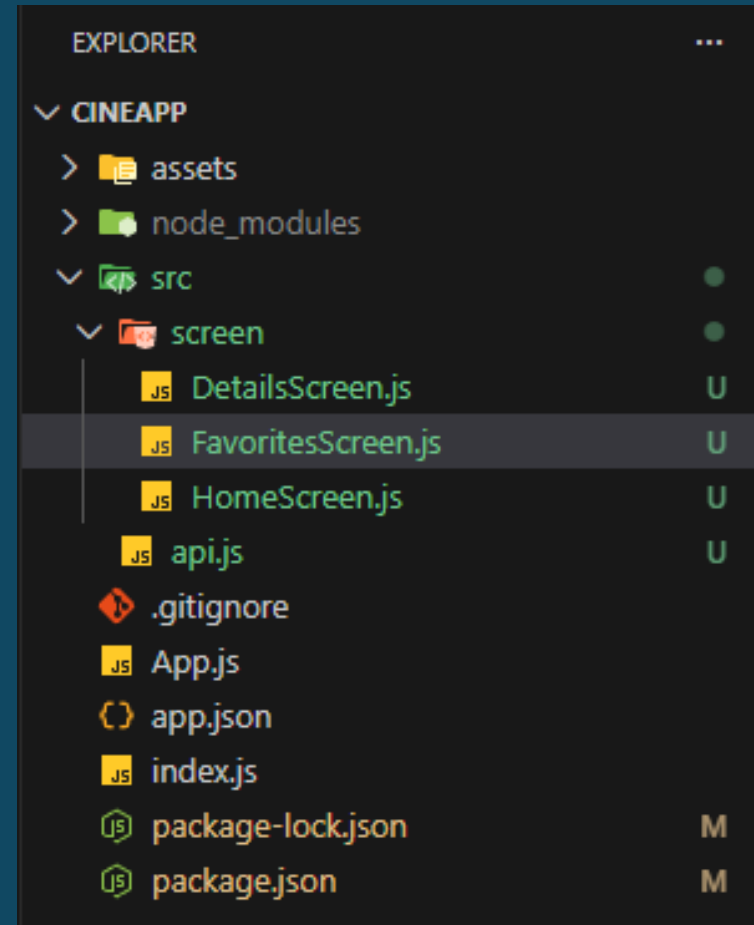
➡ O filme é um **Objeto** ({ id: 1, nome: "Matrix" }).

➡ O Storage só aceita **String**.

➡ **A Componente Lógica:** Antes de salvar, usamos JSON.stringify(listaSalva). Isso transforma o array de objetos em uma "tripa" de texto para que o celular consiga gravar.

D. Tela de Favoritos: Lendo a Persistência

- Arquivo: `src/screens/FavoritesScreen.js`





```
1 // src/screen/FavoritesScreen.js
2 import React, { useState, useEffect } from 'react';
3 import { View, Text, FlatList, Button, StyleSheet } from 'react-native';
4 import AsyncStorage from '@react-native-async-storage/async-storage';
5
6 export default function FavoritesScreen() {
7   const [favoritos, setFavoritos] = useState([]);
8
9   // Função para ler do Storage
10  const carregarFavoritos = async () => {
11    const dados = await AsyncStorage.getItem('@cinefatec_favoritos');
12    if (dados) {
13      setFavoritos(JSON.parse(dados));
14    }
15  };
16
17  // Carrega ao abrir a tela
18  useEffect(() => {
19    carregarFavoritos();
20  }, []);
21
22  const limparLista = async () => {
23    await AsyncStorage.removeItem('@cinefatec_favoritos');
24    setFavoritos([]);
25  };
26
```

```
1  return (
2    <View style={styles.container}>
3      <Text style={styles.header}>Meus Filmes Salvos</Text>
4
5      {favoritos.length === 0 && <Text>Nenhum favorito ainda.</Text>}
6
7      <FlatList
8        data={favoritos}
9        keyExtractor={item => item.id}
10       renderItem={({ item }) => (
11         <View style={styles.item}>
12           <Text>★ {item.titulo}</Text>
13         </View>
14       )}
15     />
16
17     <Button title="Limpar Favoritos" color="red" onPress={limparLista} />
18   </View>
19 );
20 }
21
22 const styles = StyleSheet.create({
23   container: { flex: 1, padding: 20 },
24   header: { fontSize: 22, fontWeight: 'bold', marginBottom: 20 },
25   item: { padding: 15, borderBottomWidth: 1, borderBottomColor: '#ccc' }
26 });
```

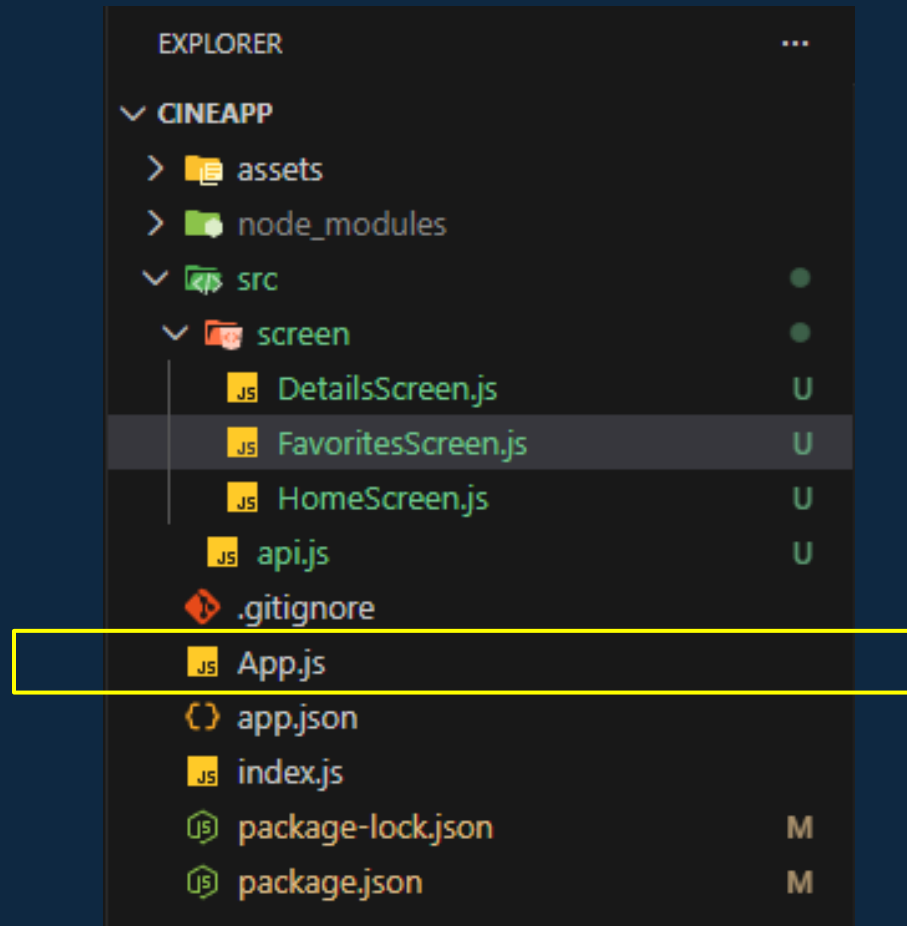


Código disponível:

<https://drive.google.com/file/d/1cUGTiSwGeUU6hfVwXByHrq5xHzJ8WS1I/view?usp=sharing>

E. O Roteador (App.js)

- Arquivo: src/screens/FavoritesScreen.js





```
1  // App.js
2  import React from 'react';
3  import { NavigationContainer } from '@react-navigation/native';
4  import { createNativeStackNavigator } from '@react-navigation/native-stack';
5
6  import HomeScreen from './src/screen/HomeScreen';
7  import DetailsScreen from './src/screen/DetailsScreen';
8  import FavoritesScreen from './src/screen/FavoritesScreen';
9
10
11 const Stack = createNativeStackNavigator();
12
13 export default function App() {
14   return (
15     <NavigationContainer>
16       <Stack.Navigator initialRouteName="Home">
17         <Stack.Screen name="Home" component={HomeScreen} options={{ title: 'Cine App' }} />
18         <Stack.Screen name="Details" component={DetailsScreen} options={{ title: 'Detalhes do Filme' }} />
19         <Stack.Screen name="Favorites" component={FavoritesScreen} options={{ title: 'Meus Favoritos' }} />
20       </Stack.Navigator>
21     </NavigationContainer>
22   );
23 }
```

Explicação:
A
Navegação
(Stack
Navigator)

- No mobile temos uma **Pilha (Stack)**.
 - **Conceito:** Imagine um baralho de cartas. A tela Home está na base. Quando clicamos num filme, colocamos a carta Details por cima. O usuário vê a carta do topo. Quando ele clica em "Voltar", removemos (pop) a carta do topo e a Home aparece novamente.
 - **No Código:** O `Stack.Navigator` gerencia esse histórico. O comando `navigation.navigate('Nome')` empilha uma tela. O comando `navigation.goBack()` desempilha.



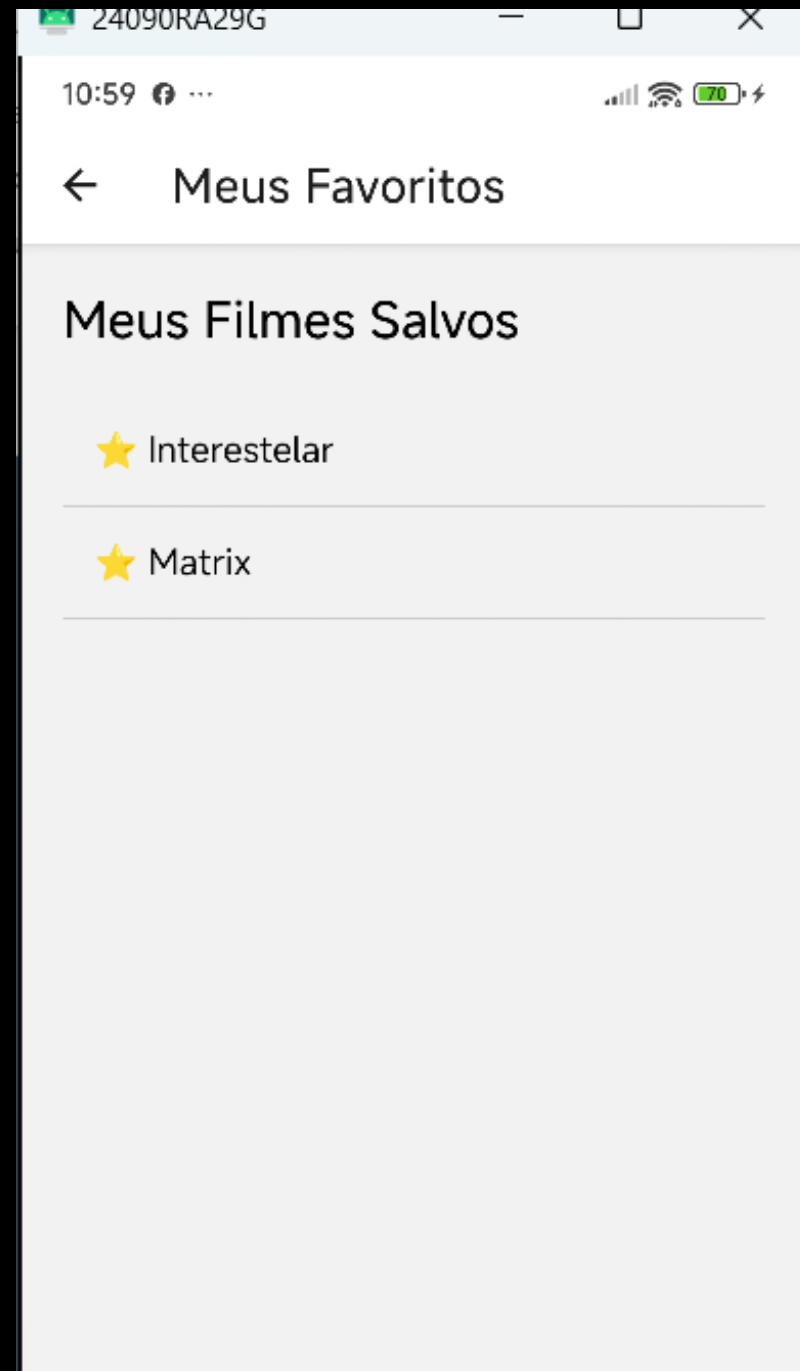
Explicação: Passagem de Parâmetros (Substituindo Intents)

- No Android Nativo (Java), usamos Intent e Bundle para passar dados. No React Native, passamos um objeto JavaScript simples como segundo argumento da navegação.
 - **Envio (HomeScreen):**
`navigation.navigate('Details', { filme: item })`
 - **Recebimento (DetailsScreen):**
`route.params.filme` Isso elimina a necessidade de serialização complexa (Parcelable) que existia no Java.



Persistência (AsyncStorage)

- O AsyncStorage é o equivalente ao **SharedPreferences** do Android ou **LocalStorage** da Web.
 - É um banco de dados **Chave-Valor** simples.
 - Só aceita **Strings**. Por isso, usamos `JSON.stringify()` para salvar objetos e `JSON.parse()` para ler objetos.
 - É **Assíncrono** (Promise), ou seja, a leitura não bloqueia a interface do usuário, garantindo que o app não trave enquanto carrega dados do disco.





Resumo da Aula

- ▶ Hoje aprendemos que um App não é uma tela estática. Ele precisa **buscar dados** (API), **navegar** levando informações (Parâmetros) e **lembrar** das escolhas do usuário (AsyncStorage). Fizemos isso usando JavaScript, substituindo as complexas Intents e SharedPreferences do Java, mas mantendo o poder de um aplicativo nativo.

Fim



Prof. Maromo